

Graph Neural Networks Powered by Encoder Embedding for Improved Node Clustering

Shiyu Chen¹, Youngser Park¹, Cencheng Shen², Carey E. Priebe¹

¹Johns Hopkins University

²University of Delaware

August 4, 2025



Joint Statistical Meetings 2025

Motivation & Goal

Motivation

- **Gap:** Vanilla Graph Neural Networks (GNNs) often converge slowly and may get trapped in sub-optimal representations when initial node features are uninformative.
- **Idea:** Use well-behaved One-Hot Graph Encoder Embeddings (GEE) [3] as an initialization “warm-start” for the GNN. Combine GEE initialization with a standard Graph Convolutional Network (GCN) [4] to form a **GEE-powered GNN (GG)**.

Goal

To achieve **faster convergence** and **superior clustering performance** on both synthetic and real-world graphs.

- [3] Cencheng Shen, Qizhe Wang, and Carey E Priebe. “One-hot graph encoder embedding”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.6 (2022), pp. 7933–7938.
- [4] Thomas N Kipf and Max Welling. “Semi-supervised classification with graph convolutional networks”. In: *arXiv preprint arXiv:1609.02907* (2016).

Method: GCN

A GCN layer

$$Z^{(\ell+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} Z^{(\ell)} W^{(\ell)}), \quad (1)$$

- $\tilde{A} = A + I_n$ adds self-loops and $\tilde{D}$ is its degree matrix.
- $W^{(\ell)}$ represents the trainable weight matrix for the ℓ -th layer.
- $\sigma(\cdot)$ denotes the activation function.
- $Z^{(\ell)}$ is the node embeddings at layer ℓ , with the **input layer** defined by $Z^{(0)} = X_v$.

Choices of initial features X_v

- randomized initial embeddings (Nearly all GNNs).
- specific node features (**We adopt GEE!**).

Final representations: $\hat{Z} = \sum_{\ell=0}^L Z^{(\ell)}$ where $L = 2$.

Loss & Prediction

Deep Modularity Networks (DMoN) loss [2]

$$\mathcal{L}_{\text{DMoN}}(C; A) = -\frac{1}{2m} \text{Tr}(C^\top B C) + \frac{\sqrt{K}}{n} \left\| \sum_i C_i^\top \right\|_F$$

- $C = \text{softmax}(\hat{Z}) \in \mathbb{R}^{n \times K}$ is the soft cluster assignment matrix.
- $m = \frac{1}{2} \sum_{i,j} A_{ij}$ is the total number of edges
- $B = A - \frac{vv^\top}{2m}$ is the modularity matrix, where v represents the degree vector of the graph.
- $\left\| \sum_i C_i^\top \right\|_F$ computes the Frobenius norm of the aggregated cluster assignments.

Goal: By minimizing this loss, we effectively **maximize the graph modularity**.

Prediction: $\hat{Y} = \arg \max_{\text{row}} C$

[2] Anton Tsitsulin et al. "Graph clustering with graph neural networks". In: *Journal of Machine Learning Research* 24.127 (2023), pp. 1–21.

The Vanilla GNN: A Random Start

GNN as a Function

The whole GNN computation can be denoted as $\hat{Y} = \mathcal{F}_{GNN}(X_v, A)$.

In the case of the vanilla GNN, we use a **randomized initial embedding** X_0 as the input of $\mathcal{F}_{GNN}$:

$$\hat{Y}_{GNN} = \mathcal{F}_{GNN}(X_0, A).$$

In our implementation, $X_0 \in R^{n \times K}$ is initialized using the **Xavier uniform initialization**, where each embedding element is sampled from $\mathcal{U}\left(-\sqrt{\frac{6}{K+n}}, \sqrt{\frac{6}{K+n}}\right)$.

Method: GEE[3]

Unsupervised GEE

The core step of unsupervised GEE is the multiplication of adjacency matrix A with a column-normalized one-hot encoding matrix W :

$$\hat{Z} = AW, \quad W_{ik} = \frac{\mathbb{I}\{\hat{Y}_i = k\}}{n_k}, \quad (2)$$

where $\mathbb{I}(\cdot)$ is the indicator function and $n_k = \sum_{j=1}^n \mathbb{I}\{\hat{Y}_j = k\}$ is the size of community $k \in [1, K]$.

We can summarize it into a function $\hat{Z} = \mathcal{F}_{GEE}(A, \hat{Y})$, where:

- **Input:** adjacency matrix A , random labels $\hat{Y}$
- **Output:** node representations $\hat{Z}$

[3] Cencheng Shen, Qizhe Wang, and Carey E Priebe. “One-hot graph encoder embedding”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.6 (2022), pp. 7933–7938.

Method: GEE

Algorithm 1 Graph Encoder Embedding Without Labels

Require: adjacency matrix A , number of clusters K , iteration limit M

Ensure: embedding $\hat{Z}_{GEE} \in \mathbb{R}^{n \times K}$, labels $\hat{Y}_{GEE} \in \{1, \dots, K\}^n$

```
1:  $\hat{Y}_{\text{new}} \leftarrow \text{RANDOMLABELS}(n, K)$  // initialize randomly
2: for  $i \leftarrow 1$  to  $M$  do
3:    $\hat{Z}_{GEE} \leftarrow \mathcal{F}_{GEE}(A, \hat{Y}_{\text{new}})$  // compute embeddings
4:    $\hat{Y}_{GEE} \leftarrow \text{KMEANS}(\hat{Z}_{GEE}, K)$  // cluster
5:   if  $\text{ARI}(\hat{Y}_{\text{new}}, \hat{Y}_{GEE}) = 1$  then
6:     break // converged
7:   else
8:      $\hat{Y}_{\text{new}} \leftarrow \hat{Y}_{GEE}$  // update labels
9:   end if
10: end for
11: return  $\hat{Z}_{GEE}, \hat{Y}_{GEE}$ 
```

Method: GEE-powered GNN (GG)[1]

- 1 **Encode:** Run unsupervised GEE on graph $G = (V, E)$ to obtain $\hat{Y}_{GEE}$ and $\hat{Z}_{GEE}$.
- 2 **Initialise:** Use $Z^{(0)} = \hat{Z}_{GEE}$ as node features in a ℓ -layer GCN.
- 3 **Train:** Fine-tune the GEE-initialized GCN end-to-end to obtain the final predictions.

$$\hat{Y}_{GG} = \mathcal{F}_{GNN}(\hat{Z}_{GEE}, A).$$

Comparison with GEE and GNN

- GEE: $\hat{Z}_{GEE}, \hat{Y}_{GEE}$ (from Algorithm 1)
- GNN: $\hat{Y}_{GNN} = \mathcal{F}_{GNN}(X_0, A)$
- GG: $\hat{Y}_{GG} = \mathcal{F}_{GNN}(\hat{Z}_{GEE}, A)$

- [1] Shiyu Chen et al. “Graph Neural Networks Powered by Encoder Embedding for Improved Node Learning”. In: *arXiv preprint arXiv:2507.11732* (2025).

Stochastic Block Model (SBM) & Simulation Setup

- **Community labels**: Each node i is assigned to a community, represented by a label $Y_i \in \{1, \dots, K\}$.
- **Edge formation**: For $i < j$

$$A_{ij} \sim \text{Bernoulli}(B_{Y_i, Y_j}), \quad A_{ji} = A_{ij}, \quad A_{ii} = 0.$$

where $B \in [0, 1]^{K \times K}$ is the *block-probability matrix*.

Parameter	Value / Range
#nodes n	800
#communities K	4
community sizes	80, 160, 240, 320
Within-community Probability p	0.15
Between-community Probability r	$[0, 0.15]$ step by 0.01
GNN embedding dim	4

Degree-Corrected SBM (DC-SBM) & Simulation Setup

- **Degree parameters**: Follow some distribution that is non trivial and bounded.
- **Edge formation**: for $i < j$

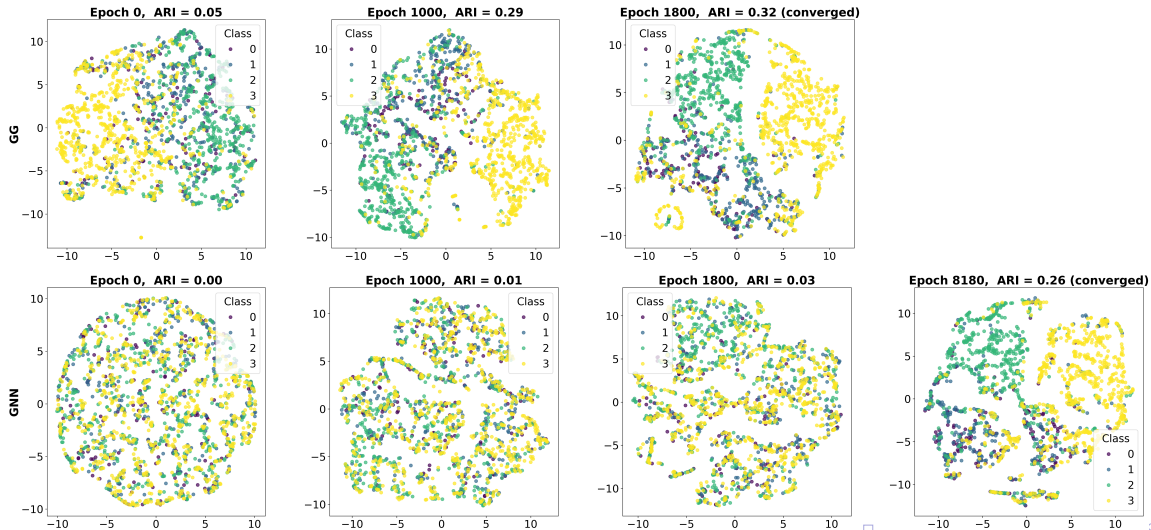
$$A_{ij} \sim \text{Bernoulli}(\theta_i \theta_j B_{Y_i, Y_j}), \quad A_{ji} = A_{ij}, \quad A_{ii} = 0,$$

where $B \in [0, 1]^{K \times K}$ is the block-probability matrix.

Parameter	Value / Range
#nodes n	2000
#communities K	4
community sizes	200, 400, 600, 800
Within-community Probability p	0.30
Between-community Probability r	[0, 0.30] step by 0.02
Degree distribution θ	Beta(1, 4)
GNN embedding dim	4

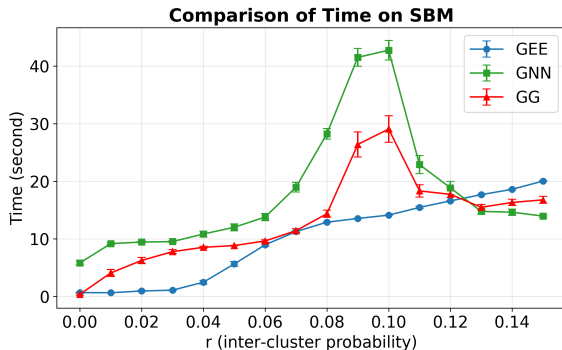
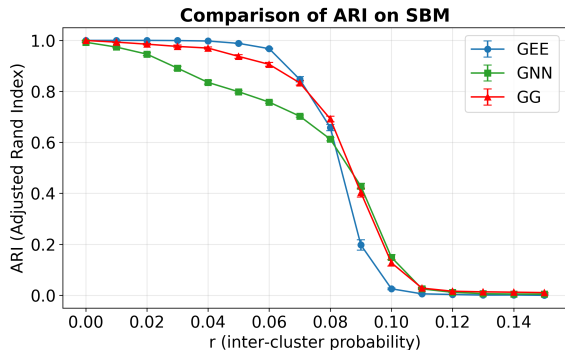
Theoretical Motivation

Training Dynamics: GG (top) vs. Vanilla GNN (bottom) on a DC-SBM graph ($r = 0.1$).



Results on SBM

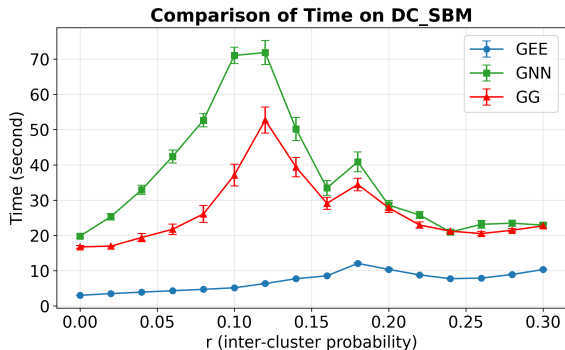
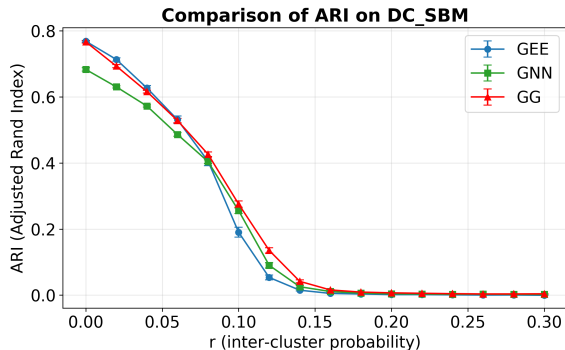
Clustering performance comparison of GEE, GNN and GG on SBM graphs.



- GG significantly **outperforms** the vanilla GNN in both **accuracy** and **speed**.
- GG also **surpasses** the classic GEE on more challenging graphs ($r > 0.08$), making it the **most effective method** overall.

Results on DC-SBM

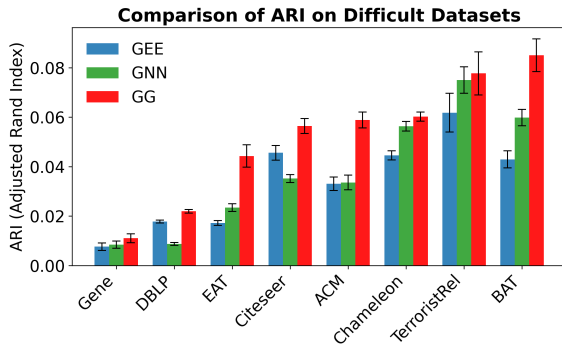
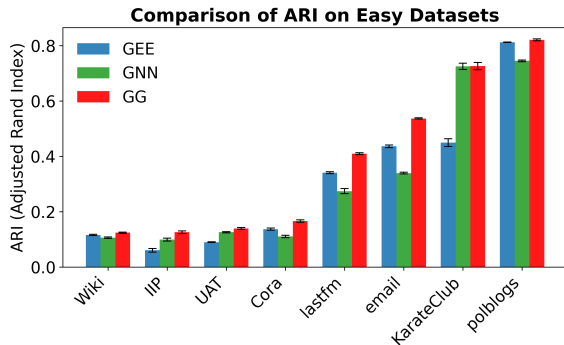
Clustering performance comparison of GEE, GNN and GG on DC-SBM graphs.



- Compared to the vanilla GNN, GG is both **more accurate** and **substantially faster** across all conditions.
- While GEE is fastest, GG **matches** or **surpasses** its accuracy, especially on more challenging graphs ($r > 0.08$).

Results on Real Data (ARI)

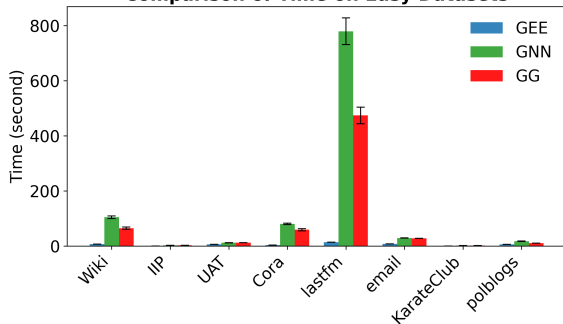
Clustering accuracy (ARI) comparison of GEE, GNN and GG on real-world datasets (**left**: easy datasets; **right**: difficult datasets).



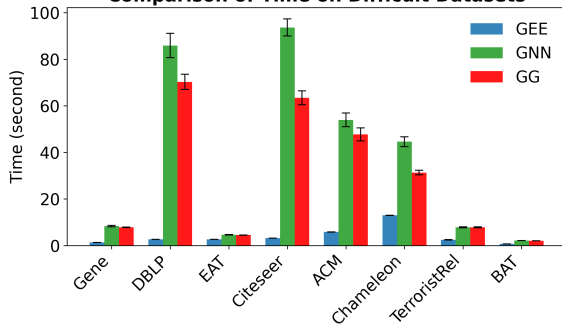
Results on Real Data (Time)

Running time comparison of GEE, GNN and GG on real-world datasets (**left**: easy datasets; **right**: difficult datasets).

Comparison of Time on Easy Datasets



Comparison of Time on Difficult Datasets



Thank You!

Q&A

- [1] Shiyu Chen et al. “Graph Neural Networks Powered by Encoder Embedding for Improved Node Learning”. In: *arXiv preprint arXiv:2507.11732* (2025).
- [2] Anton Tsitsulin et al. “Graph clustering with graph neural networks”. In: *Journal of Machine Learning Research* 24.127 (2023), pp. 1–21.
- [3] Cencheng Shen, Qizhe Wang, and Carey E Priebe. “One-hot graph encoder embedding”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.6 (2022), pp. 7933–7938.
- [4] Thomas N Kipf and Max Welling. “Semi-supervised classification with graph convolutional networks”. In: *arXiv preprint arXiv:1609.02907* (2016).

Extension: Node Classification

GEE, GNN and GG in classification

- GNN: Same GCN structure but with cross-entropy loss
- GEE: Use the supervised version of GEE and a linear discriminant analysis (LDA) for predictions.
- GG: Similar to clustering, use the GEE embedding as input for GNN training

An enhanced GG variant: GG-C

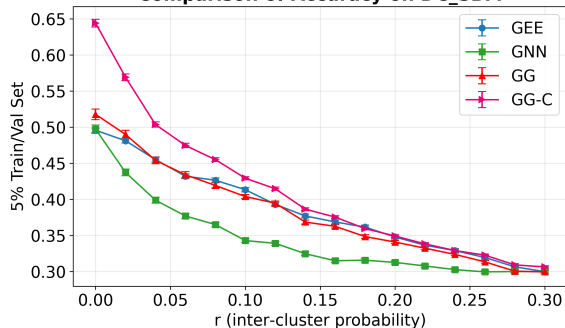
We concatenate the output of the supervised GEE, denoted as $\hat{Z}_{GEE}$, with the final GNN-refined embeddings from the GG model, denoted as $\hat{Z}_{GG}$. The concatenated embedding is then passed to an LDA for final label prediction:

$$\hat{Y}_{GG-C} = \text{LDA}([\hat{Z}_{GG}, \hat{Z}_{GEE}]).$$

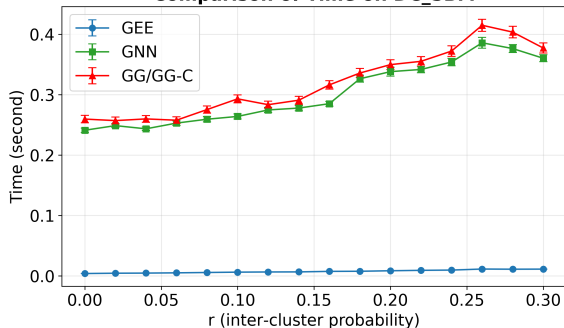
Extension: Simluation for Classification

Classification performance comparison of GEE, GNN, GG and GG-C on DC-SBM graphs with 5% train/val data.

Comparison of Accuracy on DC_SBM



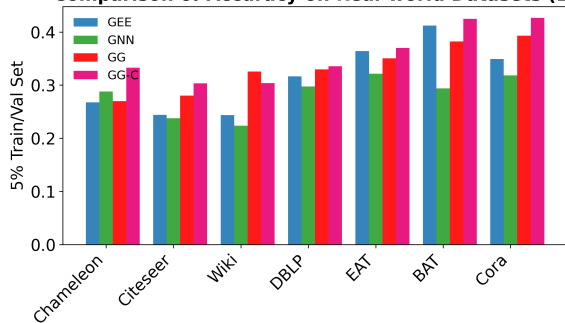
Comparison of Time on DC_SBM



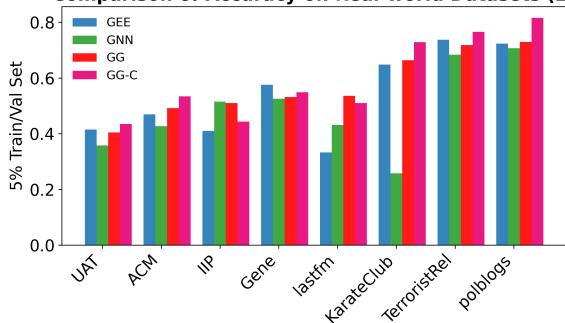
Extension: Real-data Experiments for Classification

Classification performance comparison of GEE, GNN, GG and GG-C on real-world datasets of 5% train/val data.

Comparison of Accuracy on Real-world Datasets (1)



Comparison of Accuracy on Real-world Datasets (2)



Extension: Classification Results on Real-world Datasets (1)

Table 1: Classification accuracy and running time on real-world datasets of 5% train/val data (1).

	ACM	BAT	DBLP	EAT	UAT	Wiki	Gene	IIP
Accuracy								
GEE	0.469±0.002	0.412±0.006	0.316±0.001	0.364±0.005	0.415±0.005	0.244±0.004	0.575±0.004	0.410±0.014
GNN	0.427±0.004	0.294±0.005	0.297±0.002	0.321±0.005	0.358±0.005	0.224±0.003	0.525±0.006	0.515±0.008
GG	0.492±0.005	0.382±0.007	0.329±0.002	0.351±0.005	0.404±0.006	0.326±0.003	0.531±0.005	0.510±0.010
GG-C	0.534±0.003	0.425±0.008	0.336±0.003	0.370±0.005	0.435±0.004	0.304±0.003	0.549±0.003	0.444±0.010
Running time (s)								
GEE	0.01±0.00	0.00±0.00	0.00±0.00	0.00±0.00	0.01±0.00	0.01±0.00	0.00±0.00	0.00±0.00
GNN	0.37±0.01	0.14±0.00	0.30±0.01	0.21±0.01	0.32±0.01	0.74±0.02	0.16±0.00	0.13±0.00
GG/GG-C	0.37±0.01	0.14±0.00	0.28±0.01	0.22±0.01	0.30±0.01	0.77±0.02	0.14±0.00	0.13±0.00

Extension: Classification Results on Real-world Datasets (2)

Table 2: Classification accuracy and running time on real-world datasets of 5% train/val data (2).

	LastFM	PolBlogs	TerroristRel	KarateClub	Chameleon	Cora	Citeseer
Accuracy							
GEE	0.333 ± 0.004	0.723 ± 0.005	0.737 ± 0.010	0.648 ± 0.016	0.267 ± 0.004	0.349 ± 0.006	0.244 ± 0.002
GNN	0.431 ± 0.003	0.706 ± 0.009	0.683 ± 0.006	0.257 ± 0.012	0.288 ± 0.003	0.318 ± 0.006	0.238 ± 0.003
GG	0.536 ± 0.003	0.729 ± 0.014	0.718 ± 0.011	0.663 ± 0.019	0.270 ± 0.004	0.393 ± 0.005	0.280 ± 0.003
GG-C	0.509 ± 0.003	0.815 ± 0.009	0.765 ± 0.006	0.729 ± 0.014	0.333 ± 0.003	0.426 ± 0.004	0.303 ± 0.003
Running time (s)							
GEE	0.01 ± 0.00	0.01 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.01 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GNN	1.58 ± 0.04	0.34 ± 0.01	0.26 ± 0.01	0.13 ± 0.00	0.57 ± 0.01	0.38 ± 0.01	0.36 ± 0.01
GG/GG-C	2.18 ± 0.05	0.30 ± 0.01	0.25 ± 0.01	0.13 ± 0.00	0.56 ± 0.02	0.39 ± 0.02	0.38 ± 0.01